DATAPULSE

Technical Project Report

Project direction

Hybrid observability and business-event analytics platform

Version 1.0 | 9 October 2026

Purpose
Define the problem, objectives, measurable requirements, scope, data-source strategy, and acceptance criteria before designing the system architecture.

Prepared as a practical engineering specification for a data-intensive portfolio project.

Document guide

This report establishes what DataPulse must do and how success will be measured. It deliberately does not prescribe detailed component architecture or implementation order beyond project milestones. Those decisions belong in the next architecture-design phase.

Section	Contents
1–2	Executive summary and problem statement
3–4	Objectives, intended users, and use cases
5–6	Functional and non-functional requirements
7	Data sources and canonical event contract
8–9	Technology justification and scope
10–12	Milestones, acceptance criteria, risks
13–14	Deliverables and definition of success

Project at a glance

Item	Decision
Product	Hybrid observability and business-event analytics platform
Initial data strategy	Synthetic data first; public datasets and live application telemetry later
Initial scale	At least 1 million events in a benchmark dataset; scale toward 5–10 million
Primary API stack	Python, Django, Django REST Framework, Celery
Data stores	PostgreSQL, ClickHouse, Elasticsearch; Redis for selected short-lived state and queue use
Frontend	React, Next.js, TypeScript
Operations	Sentry, Prometheus, Grafana, Loki, Tempo, Mimir
Development workflow	Docker Compose, automated tests, GitHub Actions CI

1. Executive summary

DataPulse is a data-intensive platform designed to ingest, process, search, analyse, and visualize high volumes of application telemetry and business events. It will combine operational observability—such as request latency, error rates, and failed background jobs—with business analytics, including user activity, orders, and payment events.

The project is a practical, production-oriented engineering exercise. Its purpose is not simply to connect multiple technologies, but to understand the trade-offs involved in processing large event volumes, managing asynchronous workloads, querying analytical data, building search capabilities, and operating a reliable service.

Development will begin with controlled synthetic data. Public datasets and telemetry generated by the platform itself will be introduced in later stages. The initial objective is to demonstrate correct end-to-end processing of millions of events and measure actual performance under reproducible workloads.

2. Problem statement

Modern software applications generate large volumes of operational and business data. Examples include HTTP requests, response times, status codes, application errors, background jobs, task retries, user activity, orders, payments, and other business events.

These records are often distributed across different systems and stored in formats that make unified analysis difficult. DataPulse will address the following challenges:

- **Volume:** process millions of events without excessive memory use or unacceptable latency.
- **Processing reliability:** handle retries, duplicate submissions, malformed events, and worker failures.
- **Searchability:** find events by identifiers, timestamps, text, and structured attributes.
- **Analytical performance:** calculate trends and aggregations over large datasets.
- **Operational visibility:** identify slow requests, failing jobs, growing queues, and service errors.
- **Data consistency:** account for analytical storage and search indexes updating at different times.
- **Multi-tenancy:** prevent one organization from accessing another organization's events or analytics.
- **Measurement:** distinguish actual system performance from assumed or unverified performance.

3. Project objectives

ID	Objective	Description
01	Reliable ingestion	Accept validated events through authenticated API endpoints and handle batches safely.
02	Background processing	Process large event batches outside the synchronous request-response cycle.
03	Transactional management	Use PostgreSQL for organizations, projects, access control, and ingestion-job metadata.
04	Large-scale analytics	Use ClickHouse to calculate event counts, rates, distributions, and time-based aggregations.
05	Powerful search	Use Elasticsearch to search event text and filter structured fields.
06	Interactive dashboard	Build a responsive analytics and observability interface using Next.js, React, and TypeScript.
07	Production observability	Instrument the platform to expose errors, metrics, logs, and distributed traces.
08	Performance and resilience	Benchmark throughput, latency, correctness, retries, recovery, and behavior under load.

4. Intended users and use cases

User group	Main use cases
Software engineers	Investigate application errors, inspect request latency, and trace individual requests.
Platform / operations engineers	Monitor ingestion throughput, queue delays, worker health, and processing failures.
Product / business analysts	Analyse user activity, order events, payment trends, and event volumes.

The first release will prioritize engineers and operations users. Business analytics will be represented in the event model and introduced incrementally to keep the project manageable.

5. Functional requirements

ID	Requirement	Acceptance condition
FR-01	Organization and project management	Users can create projects and access only projects they are authorized to use.
FR-02	Authentication and authorization	Protected endpoints reject unauthenticated requests and enforce project-level access control.
FR-03	Event ingestion	An authenticated client can submit a single event or a batch.
FR-04	Schema validation	Invalid event structures are rejected or reported with clear validation errors.
FR-05	Ingestion job tracking	Each accepted batch has a trackable job identifier and processing status.
FR-06	Background processing	Event processing occurs through background workers rather than lengthy HTTP requests.
FR-07	Duplicate handling	Repeated submissions with the same event identity do not silently inflate analytics.
FR-08	Failure recovery	Transient failures can be retried without duplicating completed business effects.
FR-09	Event analytics	Users can query counts, rates, and distributions over selected time windows.
FR-10	Event search	Users can search text and filter by project, event type, severity, and time range.
FR-11	Dashboard	The frontend presents ingestion health, processing status, event trends, and search results.
FR-12	Monitoring and error reporting	Developers can investigate exceptions and observe metrics, logs, and traces.
FR-13	Data retention	Events follow documented retention policies and expired data can be removed.
FR-14	Testing and CI	Automated checks run on proposed changes and failures prevent a successful CI check.

6. Non-functional requirements

The following are initial engineering targets to validate, not claims of existing performance.

Metric	Initial target	Measurement method
Dataset size	At least 1 million events; scale toward 5–10 million	Count unique persisted event IDs.
Sustained ingestion	500 events/second for 15 minutes	Load-generator throughput and accepted-event counts.
Burst ingestion	1,000 events/second for 60 seconds	Burst test with error and queue-depth measurements.
API latency	p95 below 300 ms for a batch of up to 100 small events under target load	Client-side latency measurements.
Analytics latency	p95 below 2 seconds for agreed dashboard queries	Timed queries over the benchmark dataset.
Search latency	p95 below 1 second for representative searches	Timed Elasticsearch queries.
Processing delay	95% of accepted events visible in analytics within 10 seconds under sustained target load	Compare acceptance times with query visibility.

Metric	Initial target	Measurement method
Memory behavior	No unbounded growth during a 30-minute sustained-load test	Container and process memory measurements.

Measurement rules

Record machine specifications, dataset size, batch size, concurrency, and test duration. Calculate percentile latency from measured requests, not averages. Search and analytics targets apply to defined query shapes and dataset sizes.

6.1 Reliability and data correctness

- Every accepted event must be traceable through ingestion and processing.
- Replaying an event must not create duplicate analytical counts for the same project and event identity.
- Transient failures must follow a bounded retry policy with backoff.
- Permanently invalid events must be reported clearly rather than retried indefinitely.
- A worker restart must not leave jobs permanently stuck without detection.
- The system must distinguish queued, processing, completed, partially failed, and failed batches.
- Recovery behavior must be documented for failures during writes to analytical storage and the search index.

For the learning version, the target is at-least-once task delivery with idempotent processing effects. The project will not claim true end-to-end exactly-once delivery across independent storage systems.

6.2 Security and privacy

- Enforce project and organization-level isolation on every relevant endpoint and query.
- Never store raw secrets, passwords, access tokens, or payment credentials in event payloads.
- Validate and constrain event sizes and batch sizes.
- Apply authentication, rate limiting, and input validation.
- Keep credentials in environment configuration or a suitable secrets manager rather than source control.
- Use synthetic identities and fabricated business data in initial testing.
- Define data retention and deletion behavior for all event stores.

6.3 Maintainability and delivery

- Backend code must have automated unit and integration tests.
- Frontend code must have type checks and appropriate component or integration tests.
- A new developer should be able to start the local environment using documented Docker Compose commands.
- CI must run relevant quality checks automatically.
- The project must provide reproducible load-test scripts and benchmark reports.
- Important services and processing stages must have documented operational metrics and troubleshooting guidance.

7. Data-source plan

DataPulse will use three sources in a deliberate sequence.

Phase A — Synthetic events (required for first release)

Generate controlled API, application, job-processing, user-activity, order, and payment events using Python. The generator will support configurable volume, timestamps, event types, status distributions, latency distributions, malformed records, duplicate submissions, and simulated failures.

Phase B — Public datasets (after ingestion works)

Select suitable public web-server log or application-log datasets. Confirm licensing, schema, availability, and privacy suitability before adoption. Normalize imported records to the event contract while preserving source provenance.

Phase C — Live application telemetry (after the core platform works)

Collect telemetry generated by the Django API, Celery workers, and supporting services. Use correlation IDs and distributed tracing to connect operations across components. Keep simulated telemetry distinct from telemetry generated by running software.

7.1 Initial synthetic event categories

Category	Example	Important fields
API telemetry	Endpoint returns HTTP 500	Timestamp, route, status, duration, trace ID
Application error	Exception occurs	Severity, error type, message, service
Worker telemetry	Background task fails	Task type, attempt, duration, job ID
User activity	User opens a feature	Project ID, event type, session ID, timestamp
Business events	Order completed or payment failed	Event ID, order ID, type, amount, currency
Platform health	Processing queue grows	Queue name, depth, worker count, measurement time

The generator should create realistic relationships between events. For example, an order may be created, payment attempted, and payment either succeed or fail. Events generated from a common request should be correlatable with its trace and associated background job.

7.2 Canonical event contract

Every event will follow a versioned schema. The following fields are the initial conceptual contract; required and optional fields will be finalized before implementation.

Field	Purpose
event_id	Stable identifier for deduplication.
schema_version	Allows the event format to evolve.
timestamp	Time the event occurred.
ingested_at	Time the platform accepted the event.
project_id	Project or tenant boundary.
event_type	Event category.
source	Synthetic generator, public import, or live telemetry.
service	Producing application or component.
severity	Event importance where applicable.
trace_id	Correlation with distributed traces.
attributes	Additional structured, event-specific properties.

Not every field applies to every business event. We will define required and optional fields, data types, timestamp conventions, size limits, and validation rules before implementing ingestion. Avoid unnecessary personal information; an event identifier alone must not grant access to an underlying record.

8. Technology selection and justification

Technology	Responsibility / rationale
Python	Backend and data generation; ecosystem for APIs, processing, and testing.
Django + DRF	API, authentication, validation, permissions, and business data management.
Celery	Background task execution, retries, scheduled tasks, and bounded worker concurrency.
Redis	Broker/cache and short-lived coordination where appropriate.
PostgreSQL	Transactional records, authorization, relational integrity, and job metadata.
ClickHouse	Analytical event storage, aggregation, and time-based queries.
Elasticsearch	Full-text event search and structured filtering.
React + Next.js + TypeScript	Interactive dashboards and typed frontend.
Sentry	Application exception tracking and diagnostic context.
Prometheus	Metrics collection and alerting inputs.
Grafana	Dashboards and alerting.
Loki	Log aggregation and investigation.
Tempo	Distributed tracing.
Mimir	Dedicated Prometheus-compatible metrics backend when needed.
Docker Compose	Reproducible local multi-service development.
GitHub Actions	Automated tests, quality checks, and container builds.
Cursor	AI-assisted implementation, code navigation, and debugging with human review.

The Grafana components have distinct roles. Prometheus-compatible instrumentation and exporters produce metrics; Loki stores logs; Tempo stores traces; Mimir can provide a dedicated metrics backend. Sentry complements the stack by focusing on application exceptions and diagnostic context.

Scope control: the technology list describes the intended learning scope, not a requirement to run every service on day one. Introduce tools when the milestone needs them.

9. Project scope

In scope

- Multi-project access and tenant isolation.
- Authenticated single-event and batch ingestion.
- Versioned event schema and validation.
- Background processing, retries, deduplication, and job tracking.
- PostgreSQL transactional data, ClickHouse analytics, and Elasticsearch search.
- Next.js analytics and event-explorer UI.
- Application instrumentation and monitoring with Sentry and the Grafana stack.
- Docker Compose local development, automated tests, CI checks, and reproducible load testing.
- Performance benchmarking and documented trade-offs.

Out of scope for the initial release

- Replacing enterprise observability platforms or guaranteeing zero loss under every possible infrastructure failure.
- Multi-region active-active deployment.
- A complex streaming platform before baseline needs are established.
- Machine-learning anomaly detection, billing, and commercial quota management.
- Arbitrary user-defined query languages or production collection of real customer/payment data.

10. Delivery milestones

Milestone	Work and completion focus
1. Requirements	Finalize objectives, measurable requirements, data contract, scope, risks, and acceptance criteria.
2. Architecture	Define component responsibilities, data flows, consistency guarantees, failure boundaries, and initial schemas.
3. Data and ingestion	Build the synthetic generator and validate ingestion, event identity, batch behavior, and job status.
4. Processing and storage	Add Celery processing, ClickHouse analytics, and Elasticsearch indexing with recovery and correctness tests.
5. Frontend	Build the dashboard, event explorer, processing-job view, and analytics filters.
6. Observability and CI	Integrate Sentry and Grafana stack, establish dashboards and alerts, automate CI checks and image builds.
7. Load and recovery tests	Test the million-event dataset, benchmark performance, simulate failures, and document results.

Milestones are ordered by dependency rather than fixed calendar dates. Validate each stage before introducing additional moving parts.

11. Acceptance criteria

Area	Evidence of completion
Ingestion	API accepts valid events and reports invalid input clearly.
Data integrity	Replayed events do not inflate analytical counts.
Background processing	Jobs can be tracked, retried, and recovered from worker interruptions.
Analytics	ClickHouse aggregates match a trusted test dataset.
Search	Elasticsearch returns expected matches and filters for representative queries.
Frontend	Users can inspect trends, search events, and view processing status.
Security	Automated tests demonstrate that cross-project access is denied.
Observability	An induced error can be investigated and a representative request traced across relevant services.
Performance	Million-event benchmark completes and results are recorded against defined targets.
Development workflow	Documented local startup and automated CI checks work reproducibly.
Documentation	Repository includes report, setup/testing instructions, and benchmark results.

A missed target is useful evidence. Document the bottleneck, explain the cause, make a measured change, and benchmark again rather than changing the reported result to fit the target.

12. Principal risks and mitigation

Risk	Mitigation
Excessive local resource usage	Start with a reduced Compose profile and increase service count and dataset size progressively.
Duplicate events during retries	Stable event IDs, idempotent processing, and explicit deduplication tests.
Divergent analytics and search results	Track indexing progress and document eventual consistency and replay behavior.
Database bottlenecks	Measure query plans, indexing, connection usage, and write throughput before scaling.
Growing Celery queues	Monitor queue depth and processing lag; bound concurrency and apply backpressure.
Misleading benchmarks	Fix workload parameters, record environment details, repeat tests, and report percentiles.
Overly complex first release	Implement milestones sequentially and defer optional infrastructure until needed.
Sensitive data exposure	Use fabricated data, redact secrets, enforce tenant boundaries, and define retention policies.

13. Expected project deliverables

- Technical report and requirements specification.
- System architecture document and architecture diagrams.
- Canonical event schema and synthetic-data generator.
- Working Django/DRF API and Celery processing pipeline.
- PostgreSQL, ClickHouse, and Elasticsearch data models and query examples.
- Next.js dashboard and event search interface.
- Automated backend and frontend tests.
- Docker Compose development environment.
- GitHub Actions CI pipeline.
- Sentry and Grafana-stack dashboards for errors, metrics, logs, and traces.
- Load-testing scripts, measured benchmark results, and failure-recovery reports.
- README documenting setup, design decisions, trade-offs, and limitations.

14. Definition of success

DataPulse should demonstrate understanding of the complete lifecycle of a data-intensive system:

- How events enter the system and are validated.
- How events are processed in the background and how retries and duplicate delivery are handled.
- Why transactional, analytical, and search workloads may require different storage systems.
- How the frontend retrieves and presents results.
- How to detect failures and diagnose latency.
- How to measure performance and improve it with evidence.

Next phase: system architecture
Use this report to define the high-level architecture and end-to-end data-flow diagram. Then specify component responsibilities, data movement between the API, Celery workers, PostgreSQL, ClickHouse, and Elasticsearch, failure handling, consistency guarantees, and database schemas. Architecture should follow the requirements rather than precede them.

End of report · Version 1.0